FUTURE INTERNS • CYBERSECURITY PROGRAMME • 2026

API Security Risk Analysis

FOR FUTURE INTERNS

PREPARED BY:

SATHVEEGA MITHULA

1. Executive Summary

This report presents a read-only API Security Risk Analysis conducted on the JSONPlaceholder public test API (https://jsonplaceholder.typicode.com). The assessment was performed as part of Future Interns Cybersecurity Task 3 (2026) to identify common API security vulnerabilities and communicate their business impact in clear, actionable language.

The analysis covered four key endpoints: /users, /users/{id}, /posts, and /comments. The API, being a public demo/test platform, intentionally lacks several security controls that would be critical in a real SaaS production environment. A total of 6 security risks were identified across severity levels: 3 High, 2 Medium, and 1 Low.

2. Scope & Methodology

2.1 Target API

API Name: JSONPlaceholder

Base URL: https://jsonplaceholder.typicode.com

Type: Public demo / test API

Testing Date: April 22, 2026

Tester: Sathveega Mithula

Tool Used: Postman

2.2 Endpoints Tested

- GET /users – Returns full list of all users
- GET /users/{id} – Returns individual user by ID
- GET /posts – Returns all posts
- GET /comments – Returns all comments including user email addresses

2.3 Methodology

The assessment followed a documentation-based and read-only active inspection methodology aligned with the OWASP API Security Top 10 framework. Steps included:

- Reviewed API documentation and endpoint structure
- Sent GET requests using Postman and inspected responses
- Examined HTTP response headers for security controls
- Checked for authentication/authorization requirements
- Identified excessive data exposure in API responses
- Evaluated rate-limiting, input validation and access control
- Classified each finding by severity (High / Medium / Low)

- Documented business impact and remediation guidance

2.4 Ethical Boundaries

This assessment was strictly read-only. No exploitation, denial-of-service testing, or unauthorized access was attempted. Only publicly accessible, demo-intended endpoints were tested in full compliance with ethical and legal guidelines.

3. API Testing – Postman Screenshots

The following screenshots document the actual API requests and responses captured during testing using Postman.

3.1 GET /users – Full User List Exposure

Request to /users returns all 10 user records without requiring any authentication token or API key. Each record exposes name, username, email, full address, phone, website, and company details.

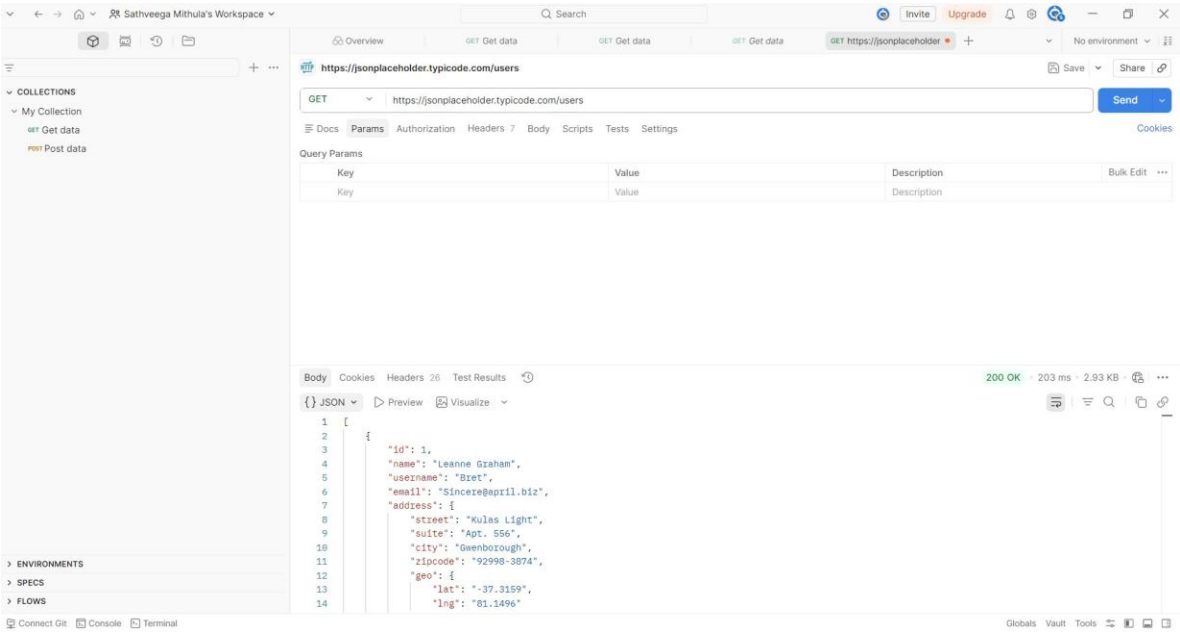

Figure 1: GET /users – Unauthenticated access returning PII for all users

3.2 GET /posts – Unrestricted Post Data

Request to /posts returns 100 post records (7.98 KB) with no authentication. The userId field in each post allows enumeration of all content belonging to any user, exposing a potential IDOR (Insecure Direct Object Reference) vulnerability pattern.

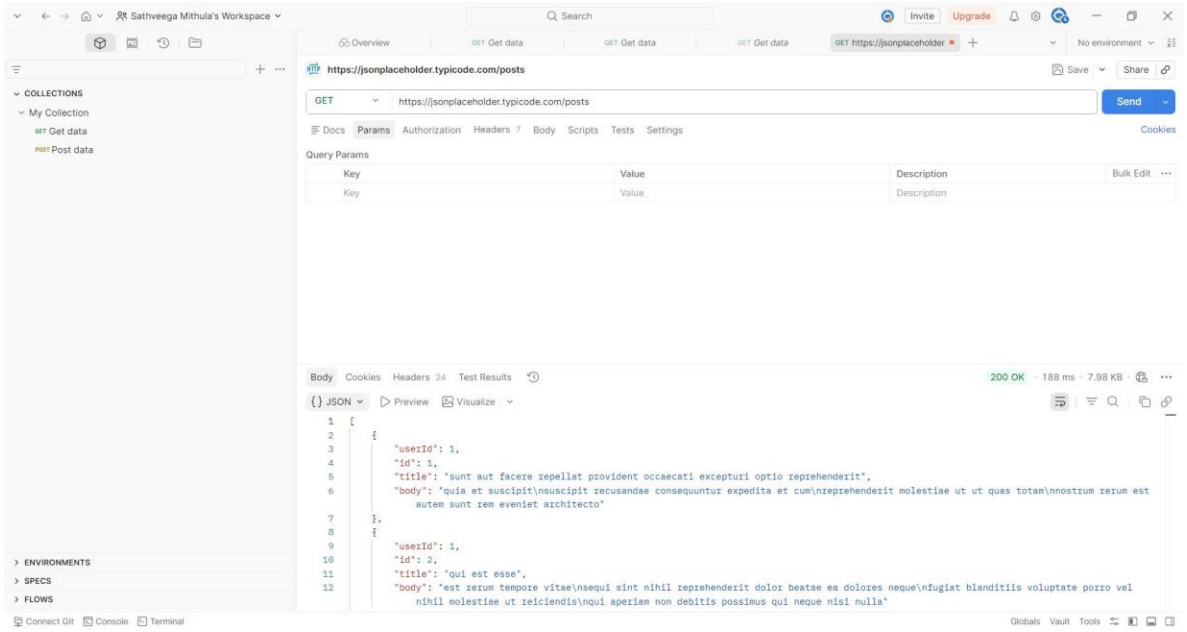

Figure 2: GET /posts – Bulk post data returned without authentication

3.3 GET /comments – Email Addresses in Public Response

The /comments endpoint returns 500 comment records (40.77 KB). Each comment includes an email field, meaning any unauthenticated caller can harvest email addresses at scale — a significant PII exposure risk.

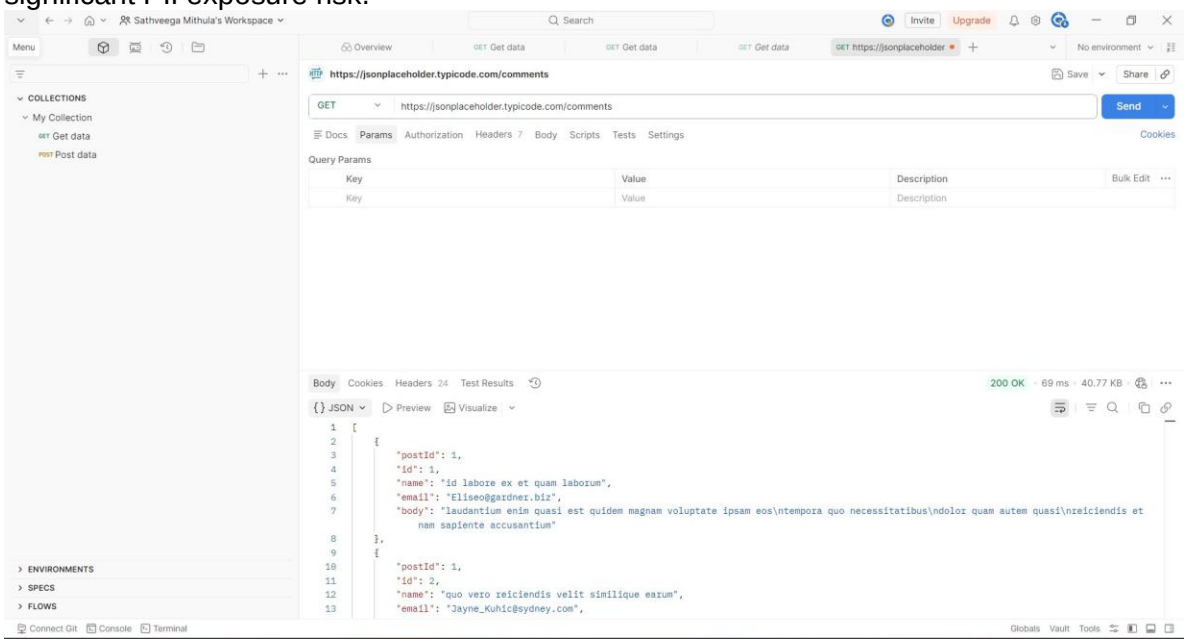

Figure 3: GET /comments – User email addresses exposed without authentication

3.4 GET /users/1 – Direct User Access by ID

The /users/{id} endpoint returns a single user's full profile by simply incrementing the numeric ID in the URL. No authentication is required, and there is no access control check, enabling full enumeration of all user accounts.

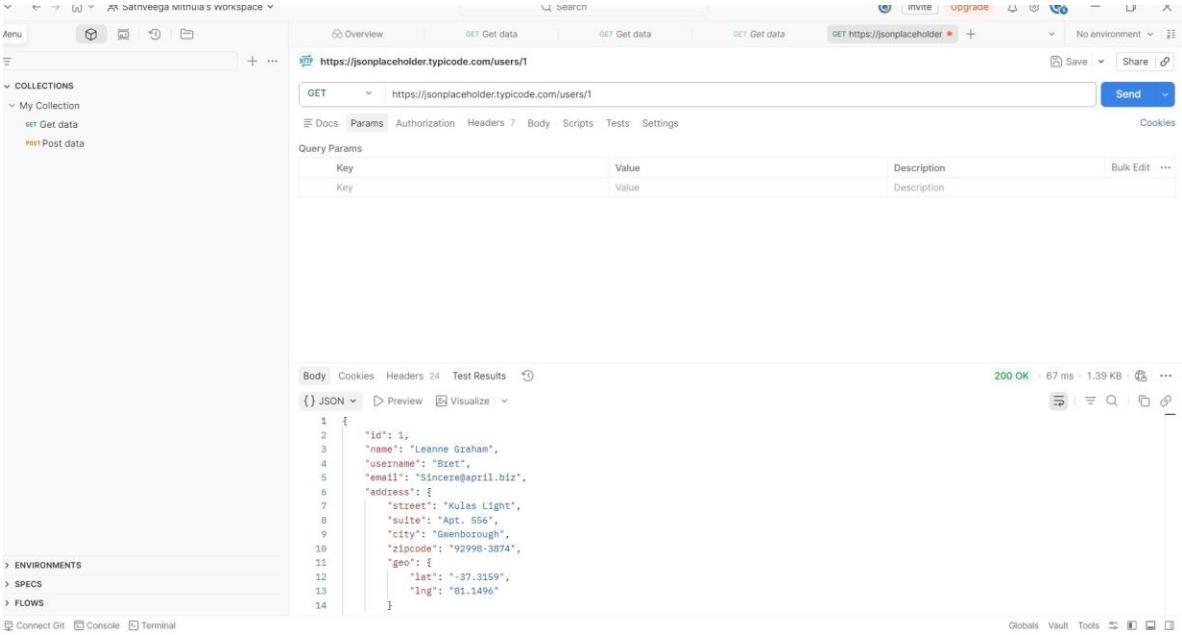

Figure 4: GET /users/1 – Direct object access with no authorization check

4. Identified Security Risks

The table below summarizes all identified risks, their severity, business impact, and recommended remediation steps.

Risk	Severity	Business Impact	Remediation
Unauthenticated Access to All Endpoints	**HIGH**	Any anonymous user can retrieve all data. In production, this would expose customer records, enable scraping, and violate data privacy laws (GDPR, PDPA).	Require authentication (JWT/OAuth 2.0) on all endpoints. Return 401 Unauthorized for unauthenticated requests.
Excessive Data Exposure – PII in Responses	**HIGH**	Full user profiles (name, email, address, phone, company) returned unnecessarily. Violates data minimization principles; increases breach impact.	Apply field-level filtering. Return only fields needed by the client. Use DTOs (Data Transfer Objects) to strip sensitive fields.
Insecure Direct Object Reference (IDOR)	**HIGH**	Users can access any account by guessing numeric IDs (e.g., /users/1, /users/2). Enables unauthorized access to other users' data.	Implement authorization checks: verify the requesting user owns or has permission to access the requested resource before returning data.
Missing Rate Limiting	**MEDIUM**	Unlimited requests allowed. Enables credential stuffing, data scraping, and denial-of-service attacks, increasing infrastructure costs.	Implement rate limiting per IP/API key (e.g., 100 req/min). Return 429 Too Many Requests when exceeded. Use tools like API Gateway or NGINX rate limiting.
Email Addresses Exposed in /comments	**MEDIUM**	500 comment records return user emails. Enables spam campaigns, phishing attacks, and targeted social engineering at scale.	Remove or hash email fields from public comment responses. Only return emails in authenticated, owner-scoped contexts.
No HTTPS Enforcement / Security Headers	**LOW**	Absence of security headers (HSTS, CSP, X-Frame-Options) increases risk of man-in-the-middle attacks and clickjacking in browser contexts.	Enforce HTTPS (HSTS header). Add Content-Security-Policy, X-Frame-Options: DENY, and X-Content-Type-Options: nosniff headers.

5. Detailed Risk Analysis

Risk 1: Unauthenticated Access to All Endpoints [HIGH]

OWASP Category: API2:2023 – Broken Authentication

Affected Endpoints: /users, /posts, /comments, /users/{id}

All four tested endpoints responded with HTTP 200 OK and full data payloads without requiring any API key, session token, or OAuth credential. In a production SaaS environment, this would mean that any attacker with knowledge of the API URL could immediately extract the entire customer database. This is the most critical risk identified in this assessment.

Risk 2: Excessive Data Exposure – PII [HIGH]

OWASP Category: API3:2023 – Broken Object Property Level Authorization

Affected Endpoints: /users, /users/{id}

The /users endpoint returns 10 full user objects, each containing: full name, username, email address, complete physical address (street, suite, city, zipcode, GPS coordinates), phone number, website URL, and company details. This far exceeds what most API consumers would legitimately need and violates the principle of data minimization required by GDPR and similar regulations.

Risk 3: Insecure Direct Object Reference (IDOR) [HIGH]

OWASP Category: API1:2023 – Broken Object Level Authorization

Affected Endpoints: /users/{id}

The /users/{id} endpoint accepts any integer ID and returns the corresponding user record with no authorization check. By simply incrementing the ID from 1 to 10, an attacker can enumerate all user records in the system. This is a classic BOLA (Broken Object Level Authorization) vulnerability — the most common API security issue according to OWASP.

Risk 4: Missing Rate Limiting [MEDIUM]

OWASP Category: API4:2023 – Unrestricted Resource Consumption

Affected Endpoints: All endpoints

No rate-limiting headers (X-RateLimit-Limit, Retry-After) were observed in any response. The /comments endpoint returned 40.77 KB in a single call with 500 records. Without rate limiting, a malicious actor could continuously scrape all data, launch enumeration attacks, or overwhelm the server, leading to service disruption and increased infrastructure costs.

Risk 5: Email Addresses Exposed in /comments [MEDIUM]

OWASP Category: API3:2023 – Broken Object Property Level Authorization

Affected Endpoints: /comments

Each of the 500 comment records contains a user email field. Comment sections are typically public-facing features, meaning emails are being returned to all clients by default. This mass exposure of email addresses enables automated harvesting for spam, phishing, or targeted social engineering campaigns — a significant privacy violation.

Risk 6: Missing Security Response Headers [LOW]

OWASP Category: API8:2023 – Security Misconfiguration

Affected Endpoints: All endpoints

HTTP response headers did not include standard security headers such as Strict-Transport-Security (HSTS), Content-Security-Policy, X-Frame-Options, or X-Content-Type-Options. While less critical for a pure JSON API, the absence of these headers in a web-accessible API increases attack surface for clients consuming the API in browser contexts.

6. Risk Summary

Severity	Count
HIGH	3
MEDIUM	2
LOW	1
TOTAL	6

7. Key Recommendations

The following remediation priorities are recommended for any production API based on these findings:

1. **Implement Authentication Immediately**

Deploy OAuth 2.0 or JWT-based authentication across all endpoints. All requests must include a valid bearer token; unauthenticated requests should return HTTP 401.

2. **Fix IDOR with Authorization Checks**

Before returning any resource by ID, verify the requesting user is authorized to access that specific resource. Never rely on security by obscurity.

3. **Apply Data Minimization to All Responses**

Return only the fields required for the client's use case. Remove GPS coordinates, physical addresses, and company details from general-purpose endpoints. Strip emails from comment responses.

4. **Enforce Rate Limiting at API Gateway Level**

Configure rate limits per authenticated user/IP. Return proper HTTP 429 responses with Retry-After headers. Consider tiered limits for different user roles.

5. **Add Standard Security Headers**

Enable HSTS, CSP, X-Frame-Options, and X-Content-Type-Options on all API responses via server or reverse-proxy configuration.

8. Conclusion

This API security risk analysis of the JSONPlaceholder test API identified 6 vulnerabilities, 3 of which are rated High severity. While JSONPlaceholder is intentionally open as a public testing API, these findings illustrate the exact patterns that exist in poorly secured production SaaS APIs — the most common and costly attack vectors in modern application security.

If these vulnerabilities existed in a live production environment, the consequences would include unauthorized access to customer data, regulatory fines under GDPR/PDPA, reputational damage, and potential exploitation by malicious actors. Addressing the High severity findings — authentication, IDOR, and data exposure — should be the immediate priority for any development team.

This analysis demonstrates the core skill set of an API Security Consultant: systematic endpoint testing, risk identification, impact assessment, and clear remediation guidance using industry-standard frameworks (OWASP API Security Top 10).

9. References

- OWASP API Security Top 10 (2023): https://owasp.org/API-Security/editions/2023/en/0x00-header/
- JSONPlaceholder API: https://jsonplaceholder.typicode.com
- API Security Checklist (Shieldfy): https://github.com/shieldfy/API-Security-Checklist
- Postman API Testing Tool: https://www.postman.com
- Public APIs Collection: https://github.com/public-apis/public-apis